



Amazon MQ

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Amazon MQ



Amazon MQ is a managed message broker service for the opensource projects Apache ActiveMQ and RabbitMQ



Active MQ is a powerful open-source messaging server that supports a wide range of protocols including AMQP, MQTT, STOMP, and JMS, offering robust features for JMS-centric, enterprise-integrated messaging scenarios.

Amazon MQ ActiveMQ supports: JMS, NMS, AMQP 1.0 STOMP, MQTT, and WebSocket



RabbitMQ is a highly reliable, scalable, and flexible messaging broker that supports advanced messaging protocols like AMQP, MQTT, and STOMP, making it ideal for complex routing scenarios and high-throughput requirements.

Amazon MQ RabbitMQ supports: AMQP 0-9-1



Amazon MQ has a similar offering to SQS however MQ can handle more complex delivery rules with different performance guarantees

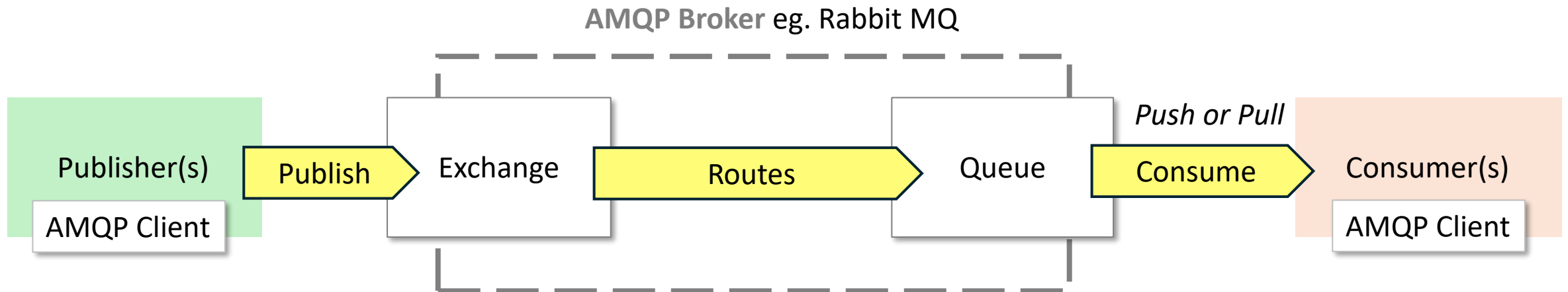
Amazon MQ

	Rabbit MQ	Active MQ
Protocol Support	AMQP, MQTT, STOMP	AMQP, MQTT, STOMP, OpenWire
Performance	Generally considered faster and more efficient in scenarios with lightweight messages and high throughput.	Good performance, Slower under high load compared to RabbitMQ. Offers more features which can add overhead.
Message Routing	Advanced message routing capabilities with exchange types like direct, topic, headers, and fanout.	Basic message routing capabilities with more emphasis on JMS (Java Message Service) standards.
Transaction Support	Basic transaction support.	Advanced support for JMS transactions.
Flexibility Support	Highly flexible due to support for multiple messaging protocols and extensive client library support.	Flexible, especially within Java environments due to JMS. Offers a broad range of connectors for different systems.

AMQP



AMQP (Advanced Message Queuing Protocol) is an **open standard wire-level protocol** designed for messaging middleware that enables conforming client applications to communicate with conforming messaging middleware servers



- **Messages** are published to **Exchanges**
- **Exchanges** distribute message copies to **Queues** using rules called **Bindings**
- The **Broker** pushes messages to the subscribed consumers
 - *Or the consumers pull messages from the queues*
- Messages can have **metadata** attached to them
- Messages are only removed from the queue when a consumer ACKS the broker
 - ACK is short for acknowledgement.

There are four types of exchanges

- Direct exchange (default)
- Fanout exchange
- Topic exchange
- Headers exchange

AMQP

AWS Ruby SDK has a library called bunny which is RabbitMQ client that uses the AMQP protocol.

- Create a channel
- Create or set a queue
- Get an exchange
- Publish a message

***Publishing** to a queue example*



```
require 'bunny'

# Start a a connection to RabbitMQ
connection = Bunny.new
connection.start

channel = connection.create_channel
queue = channel.queue('hello')
exchange = channel.default_exchange

exchange.publish('Hello, world!', routing_key: queue.name)
puts " [x] Sent 'Hello, world!'"

connection.close
```

AMQP

***Subscribing** to a queue example in Ruby for AMQP for RabbitMQ*



```
require 'bunny'

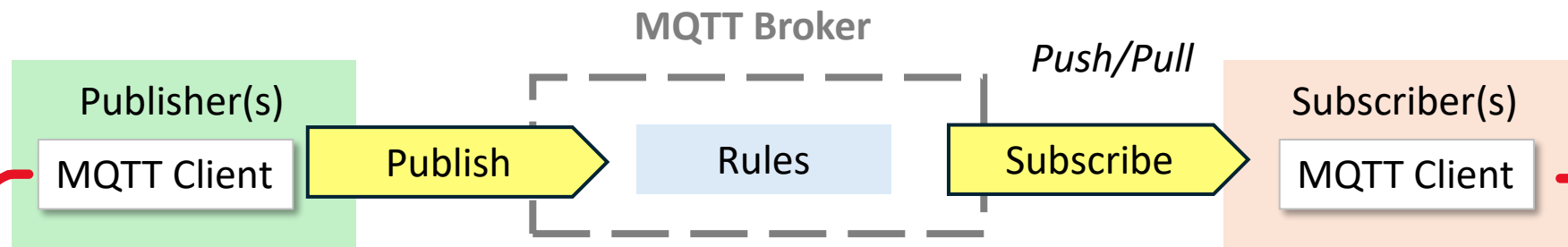
connection = Bunny.new
connection.start
channel = connection.create_channel
queue = channel.queue('hello')

begin
  puts ' [*] Waiting for messages. To exit press CTRL+C'
  queue.subscribe(block: true) do |_delivery_info, _properties, body|
    puts " [x] Received #{body}"
  end
rescue Interrupt => _
  channel.close
  connection.close
  exit(0)
end
```

MQTT



MQTT (MQ Telemetry Transport) is a light-weight pub/sub messaging protocol. MQTT uses minimal network bandwidth and is often used in Internet of Things (IoT) or real-time messaging apps. Suitable for machine-to-machine (M2M) communication.



```
require 'mqtt'

host = 'localhost'
topic = 'test/topic'
message = 'Hello MQTT'

begin
  MQTT::Client.connect(host) do |client|
    client.publish(topic, message)
  end
rescue => e
  puts "Failed: #{e.message}"
end
```

- To publish or subscribe you use the **MQTT client** for your programming language
- A topic name is used to publish and subscribe eg. /hello/5/world

```
require 'mqtt'

host = 'localhost'
topic = 'test/topic'

begin
  MQTT::Client.connect(host) do |client|
    client.get(topic) do |topic, msg|
      puts topic
      puts msg
    end
  end
rescue => e
  puts "Failed: #{e.message}"
end
```

STOMP



STOMP (Simple Text Oriented Messaging Protocol) is a **simple, text-based wire-protocol** that allows clients to communicate with almost any message broker.



STOMP is so simple that it **can be used with a telnet client.**

```
require 'stomp'

client = Stomp::Client.new({
  hosts: [{
    login: 'guest', passcode: 'guest',
    host: 'localhost', port: 61613,
    ssl: false
  }]
})

# (queue or topic)
dest = "/queue/test"

# Publish a message
client.publish(dest, 'Hello, World!')
puts "Published message to #{dest}"
client.close
```

Publish



Subscribe



```
require 'stomp'

client = Stomp::Client.new({
  hosts: [{
    login: 'guest', passcode: 'guest',
    host: 'localhost', port: 61613,
    ssl: false
  }]
})

# (queue or topic)
dest = "/queue/test"

client.subscribe(dest) do |message|
  puts "Received: #{message.body}"
  client.acknowledge(message)
end

puts "Subscribed to #{dest}."
puts "Press CTRL+C to exit."
sleep
```